

Internship Report

Autonomous Local Data Analyst

S Sriram

The Problem

- Business users usually know what they want to ask about data, but not how to write SQL for it.
- Tools where an AI silently writes and runs code end-to-end are a black box — if the answer looks wrong, there is no way to check why.
- Nothing reusable is left behind: the next person cannot re-run, verify, or build on a chat answer.
- Goal: let a user ask in plain English, but keep the actual computation inside a proper, auditable BI engine — never inside the AI itself.

Design Principle: Who Does What

Two systems, two separate jobs:

- **Apache Superset** is the analyst — it owns data connections, runs every query, and renders every chart and dashboard.
- **Local LLMs** are the assistant — they only translate a question into SQL and explain results computed in Superset.
 1. The LLM never touches, computes, or stores the actual data itself.
 2. This separation means every answer can be independently verified, re-run, or shared — because it exists as a real Superset query, not just a chat message.

Analytical Core: Superset & DuckDB

- **Apache Superset:** open-source BI platform; provides dashboarding, chart building, and a production-style SQL Lab for direct querying.
- **DuckDB:** an embedded, file-based analytical database — no server setup, extremely fast for read-heavy analytical queries, ideal for local multi-file joins Superset is the workbench, DuckDB is the engine underneath doing the actual number-crunching.

AI-Driven Reasoning & Support

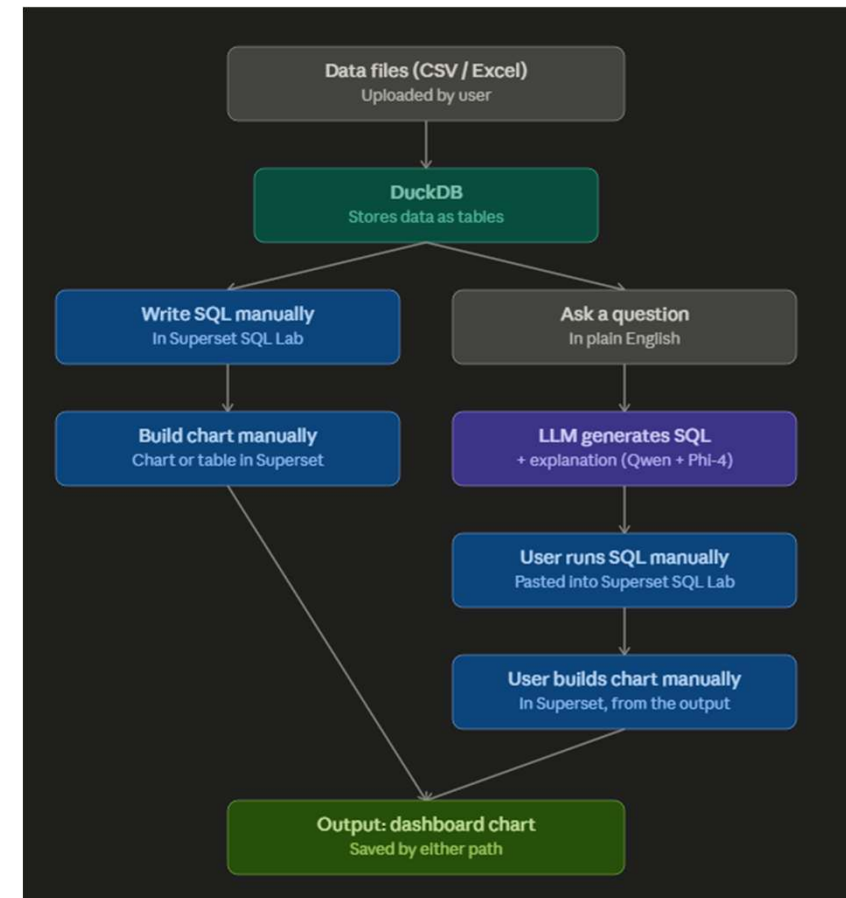
Role: convert a natural-language question into a correct, DuckDB-compatible SQL query — nothing else.

Two specialised local models, run via Ollama (no external API calls, data stays local):

- **qwen2.5-coder-7b-instruct** — writes precise SQL from the question and table schemas.
- **phi4-mini-reasoning** — reads the SQL result and explains it, including the business implication.

End-to-End Pipeline

- 1.Ingestion** — CSV/Excel files are loaded and indexed as tables inside DuckDB.
- 2.Exploration** — user can query and build charts manually in Superset.
- 3.Augmentation** — for help, the user asks the LLM Copilot a question in plain English.
- 4.Translation** — the LLM converts that question into one SQL query.
- 5.Execution** — Superset's SQL Lab runs the query against DuckDB (logged, auditable). If the user wishes, they can create a chart with the query output and add it to dashboard.
- 6.Explanation** — the LLM reads only the returned result table and writes a plain-language answer.



Olist E-Commerce Dataset

Dataset Link: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

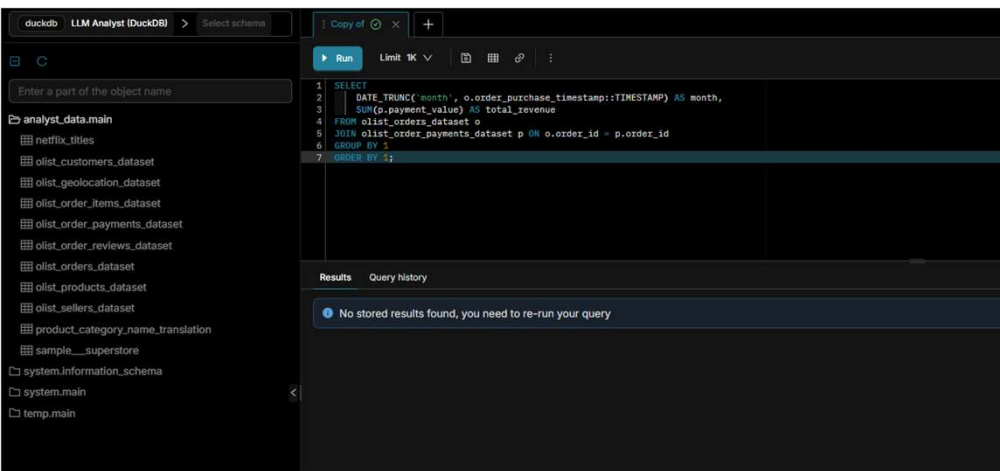
A real-world, multi-table e-commerce dataset from Olist, a Brazilian marketplace: roughly 100,000 orders placed between 2016 and 2018, spanning orders, payments, customers, products, sellers, and reviews. Used here to test multi-table joins and business-style questions such as revenue by category, payment method mix, and delivery performance.

Data Ingestion Interface

This screen shows the ingestion portal where users can upload multiple files simultaneously. The system automatically syncs these files into DuckDB, making them ready for immediate querying, as shown by the 'Ready!' status confirming all tables are loaded.

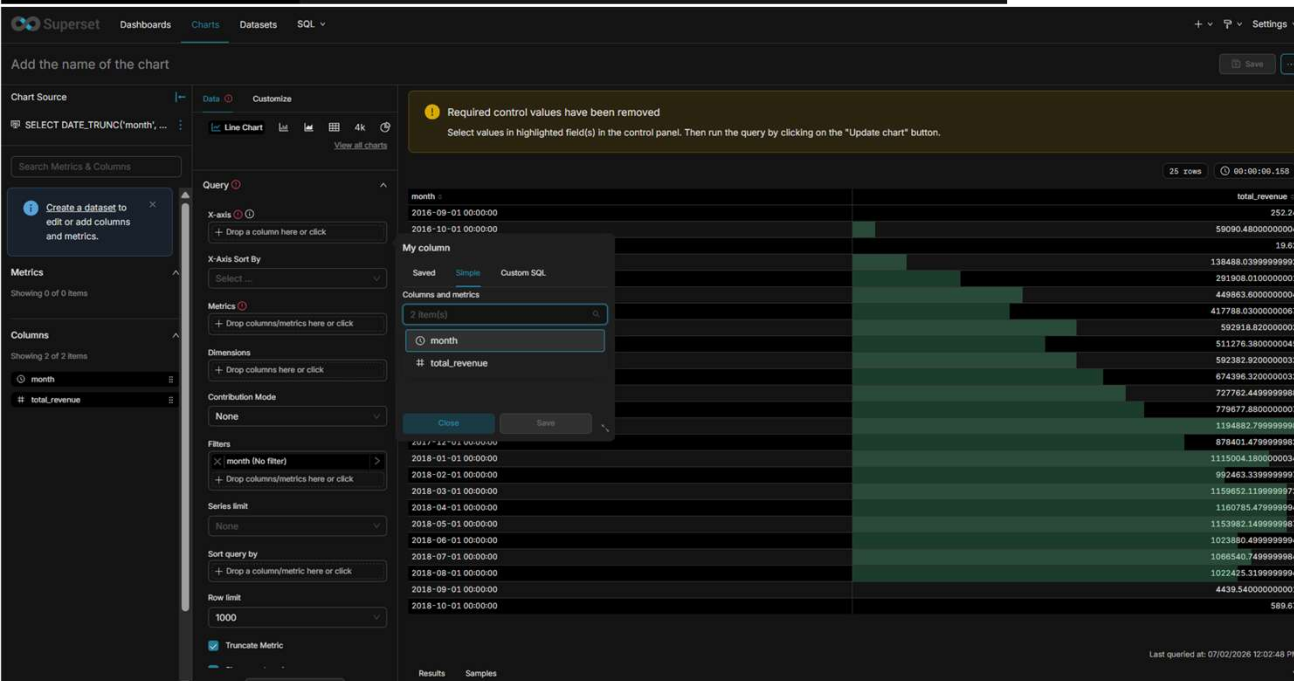
The screenshot displays the 'Autonomous Data Analyst — BI-Backed' interface. At the top, it indicates analysis by Apache Superset, explained by phi4-mini-reasoning and qwen2.5-coder:7b-instruct. The 'Attach Data' section shows three CSV files uploaded: olist_customers_dataset.csv (8.6 MB), olist_geolocation_dataset.csv (58.4 MB), and olist_orders_dataset.csv (16.8 MB). A green status message confirms: 'Ready! Loaded 9 table(s) from the folder and synced to DuckDB.' Below this, a link to 'Open in Superset SQL Lab' is provided with the URL <http://localhost:8088/sqlab/>. A table lists the loaded datasets with their row and column counts. At the bottom, there is a 'Textbox' for asking questions about the data.

Table (in DuckDB / Superset)	Rows	Cols
olist_customers_dataset	99,441	5
olist_geolocation_dataset	1,000,163	5
olist_orders_dataset	99,441	8
olist_order_items_dataset	112,650	7
olist_order_payments_dataset	103,886	5
olist_order_reviews_dataset	99,224	7
olist_products_dataset	32,951	9
olist_sellers_dataset	3,095	4
product_category_name_translation	71	2



Manual Querying on database in Superset:

Users in Superset can analyze and query the data using an SQL engine. The output of the query can be converted into a chart of the user's choice.

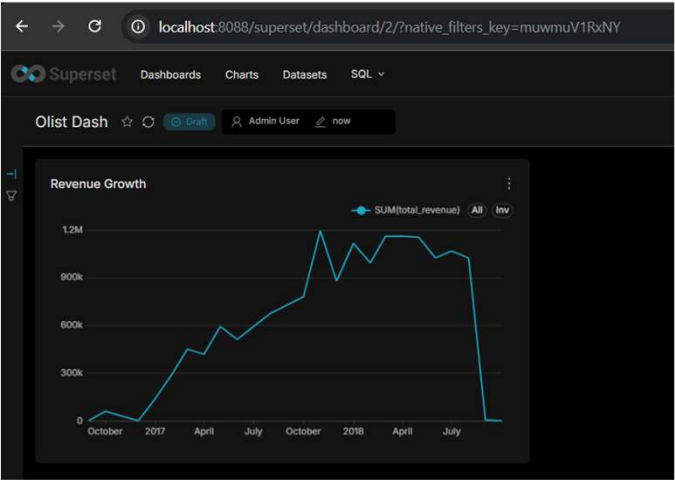


Creating a chart with output of SQL queries:

In this image the user is creating a line chart between month and total revenue manually.

Line chart output:

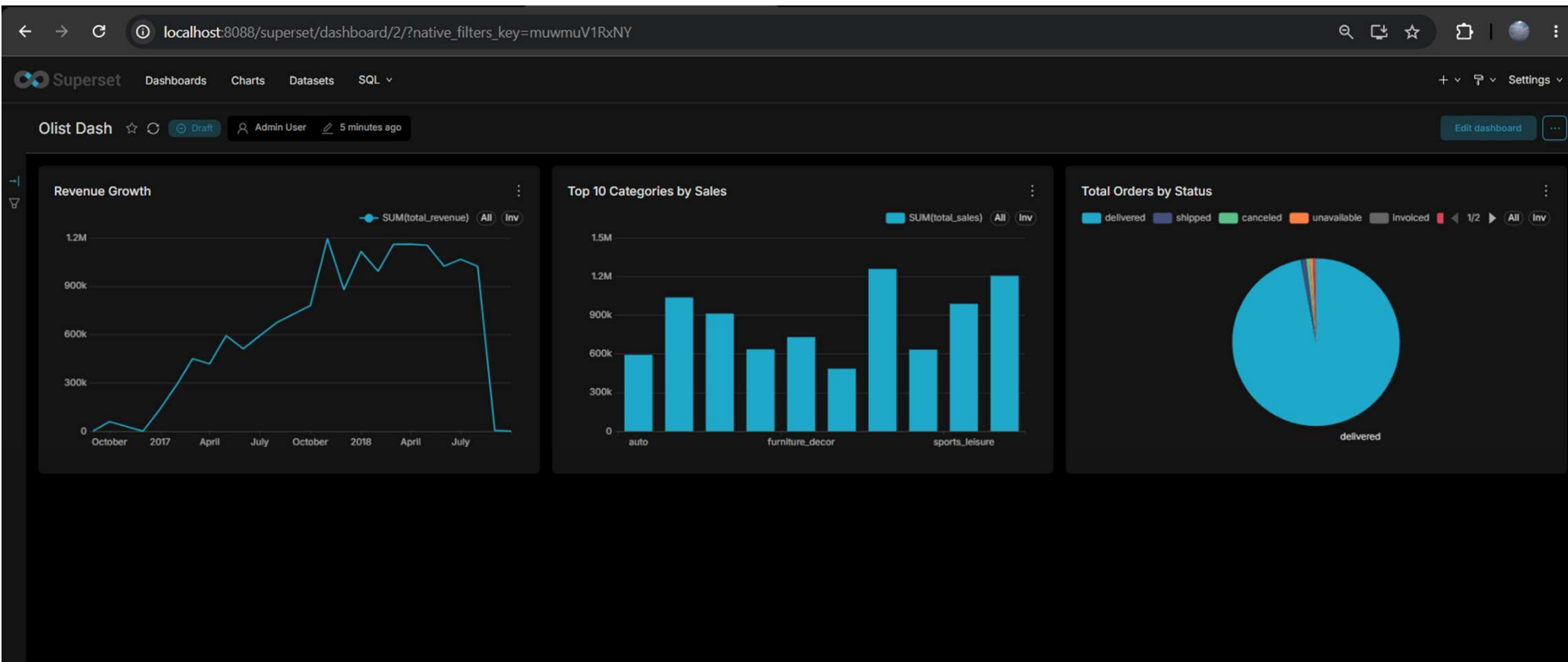
The below image shows the output of the line chart created. This chart has also been saved to a Dashboard called Olist Dash.



Bar chart manually created on Superset:

Similarly we've manually configured bar chart ranking 'Top 10 Categories by Sales.'





Olist Dashboard Visualization:

This is a finished dashboard widget showing high-level view of business performance. The 3 charts in this dashboard have been created manually by the user in Superstore itself.

localhost:7860

Ready! Loaded 9 table(s) from the folder and synced to DuckDB.

You: Which 5 states have the highest number of customers? And for those 5 states, what is the total number of orders placed?

Analyst - Answer: The top 5 states are SP (41,746), RJ (12,852), MG (11,635), RS (5,466), and PR (5,045), totaling 76,744 orders.

- **Analysis:** By sorting the aggregated order counts in descending order and selecting the first five rows, we identify the states with the highest customer orders. Summing their "total_orders" values yields the combined total.
- **Business Implication:** Focusing marketing efforts or resource allocation on these high-order states could maximize revenue potential, as they represent the largest customer bases within the dataset.

Open in Superset SQL Lab: <http://localhost:8088/sqlab/>

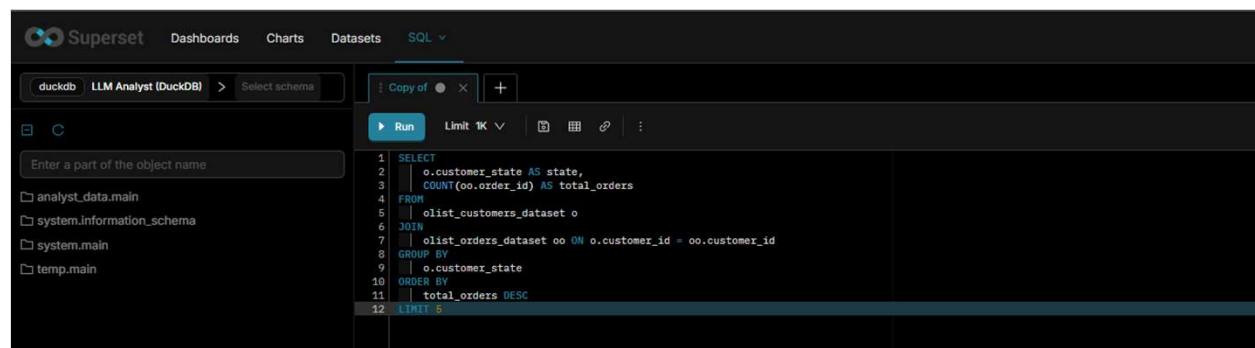
Table (in DuckDB / Superset)	Rows	Cols
olist_customers_dataset	99,441	5
olist_geolocation_dataset	1,000,163	5
olist_orders_dataset	99,441	8
olist_order_items_dataset	112,650	7
olist_order_payments_dataset	103,886	5
olist_order_reviews_dataset	99,224	7
olist_products_dataset	32,951	9
olist_sellers_dataset	3,095	4
product_category_name_translation	71	2

```
SELECT
  o.customer_state AS state,
  COUNT(oo.order_id) AS total_orders
FROM
  olist_customers_dataset o
JOIN
  olist_orders_dataset oo ON o.customer_id = oo.customer_id
GROUP BY
  o.customer_state
ORDER BY
  total_orders DESC
LIMIT 6
```

state	total_orders
SP	41746
RJ	12852

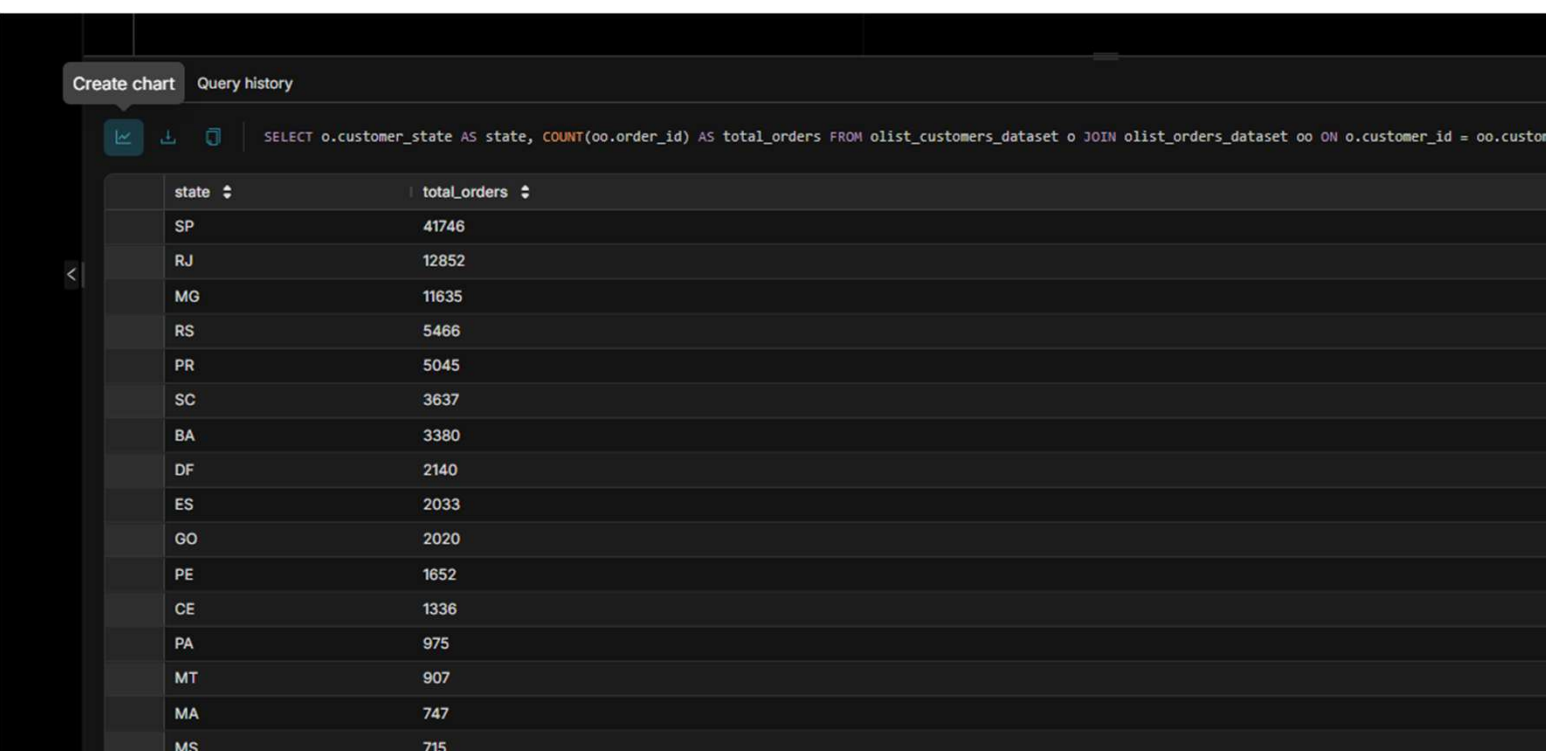
LLM assisted querying:

While the dashboard till now has been created manually, the gradio app used to upload and load the files also contains a local LLM pipeline that can process queries in natural language and give explanation and SQL codes which can be run in the SQL lab in Superset to get desired results.



Running LLM generated SQL script on Superset SQL lab:

LLM generated logic is run in SQL lab to get output and we have an option to convert the output to a chart of our choice.



Output to chart:

The image shows the use of create chart option to represent query outputs as charts.

Chart Source: SELECT o.customer_state AS st...

Query mode: Aggregate (selected), Raw records

Columns: state, total_orders

Filters: + Drop columns/metrics here or click

Ordering: total_orders [desc]

Server pagination: ☐

Row limit: 1000

Results: 5 rows, 00:00:00.132

state	total_orders
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045

Last queried at: 07/02/2026 12:33:31 PM

Save chart

☐ Save (Overwrite) ☒ Save as...

Chart name: Top 5 states by orders

Dataset Name: Olist 4

Add to dashboard: Olist Dash

Add to tabs: Select a tab

Buttons: Cancel, Save & go to dashboard, Save

Table creation:

The user in the above image has decided to represent the Top 5 states by total orders in the form of a table and has saved the table to the main Olist Dashboard.

 **You:** What are the most popular payment methods used by our customers? Please show the total number of orders and the total revenue generated for each payment type.

 **Analyst: Answer:**

The most popular payment method is **Credit Card**, which processed 76,795 orders and generated ₦12,542,100 in revenue.

Analysis:

- **Voucher:** 5,775 orders, ₦379,437.
- **Boleto:** 19,784 orders, ~₦2,869,360.00.
- **Credit Card:** Highest in both metrics (76,795 orders, ₦12,542,100).
- **Debit Card:** 1,529 orders, ₦218,990.
- **Not Defined:** Only 3 orders and ₦0 revenue.

Business Implication:

Focusing on credit card processing could maximize revenue due to its dominance in both transaction volume and value. Expanding payment options for credit cards or optimizing their acceptance may further strengthen revenue streams. Boleto, while high in order count, underperforms financially compared to credit cards. This insight suggests prioritizing credit card partnerships or incentives to capitalize on its strong market presence.

Textbox

Ask about your data...

Send

Clear

 Open in Superset SQL Lab: <http://localhost:8088/sqllab/>

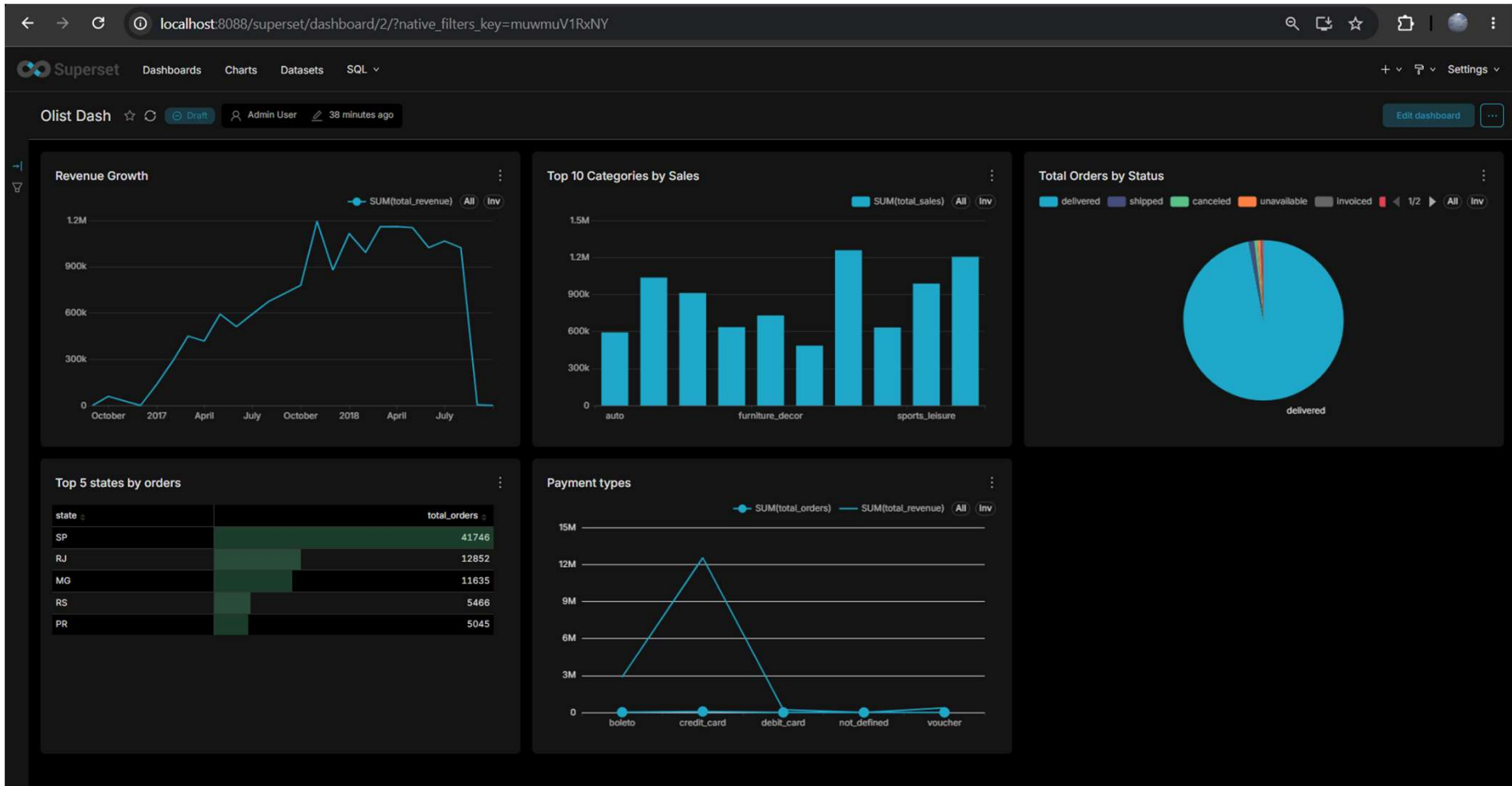
Table (in DuckDB / Superset)	Rows	Cols
<code>olist_customers_dataset</code>	99,441	5
<code>olist_geolocation_dataset</code>	1,000,163	5
<code>olist_orders_dataset</code>	99,441	8
<code>olist_order_items_dataset</code>	112,650	7
<code>olist_order_payments_dataset</code>	103,886	5
<code>olist_order_reviews_dataset</code>	99,224	7
<code>olist_products_dataset</code>	32,951	9
<code>olist_sellers_dataset</code>	3,095	4
<code>product_category_name_translation</code>	71	2

```
SELECT
  op.payment_type,
  COUNT(oo.order_id) AS total_orders,
  SUM(op.payment_value) AS total_revenue
FROM
  olist_order_payments_dataset op
JOIN
  olist_orders_dataset oo ON op.order_id = oo.order_id
GROUP BY
  op.payment_type
```

payment_type	total_orders	total_revenue
voucher	5775	379437
debit_card	1529	217990
not_defined	3	0
boleto	19784	2.86936e+06

LLM assisted query:

Similar to the previous case we've run the LLM to assist us to get most popular payment methods and the LLM pipeline has given us the SQL code for Superset along with own analysis of the results.



Complete Olist Dashboard

The fully populated Olist Dashboard, combining both manual and LLM-generated charts. It demonstrates how the tool centralizes diverse metrics—from geography to payment efficiency—in one hub.

Indian Weather Dataset

Dataset link: <https://www.kaggle.com/datasets/pratikjadhav05/indian-weather-data>

The dataset is a comprehensive collection of historical weather data for various locations in India. It includes information such as temperature (in Celsius and Fahrenheit), air quality metrics, precipitation levels, wind speed and direction, humidity, and weather conditions. The dataset covers multiple locations in India and provides data on a daily or monthly basis. This rich dataset offers valuable insights into past weather patterns and air quality conditions, making it a valuable resource for informed decision-making and in-depth analysis.

Indian weather data dashboard:

This dashboard was created on an Indian weather dataset and the table and scatter plot were manually created on Superset using SQL queries and chart options (just like we did at the beginning in Olist).



You: Does higher temperature correspond to higher levels of ozone?

Analyst - Answer: No, higher temperature does not consistently correlate with higher ozone levels.

- Analysis:** While some lower temperatures (e.g., 35.6°C) show elevated ozone values compared to slightly cooler highs (36.3°C/38.3°C), the relationship is inconsistent—some low-temperature rows have both high and low ozone, indicating no clear trend.
- Business Implication:** Decision-makers should not assume temperature directly drives ozone levels; other factors like geography or time of day likely influence readings. Policies targeting air quality must account for this variability rather than relying solely on temperature correlations.

Open in Superset SQL Lab: <http://localhost:8088/superset/>

Table (in DuckDB / Superset)	Rows	Cols
air_quality_information	24,070	9
astronomical	24,070	7
location_information	24,070	8
weather_data	24,070	20

```
SELECT
  w.temperature_celsius AS temperature,
  a.air_quality_Ozone AS ozone
FROM
  weather_data w
JOIN
  air_quality_information a ON w.last_updated_epoch = a.last_updated_epoch
ORDER BY
  temperature DESC
```

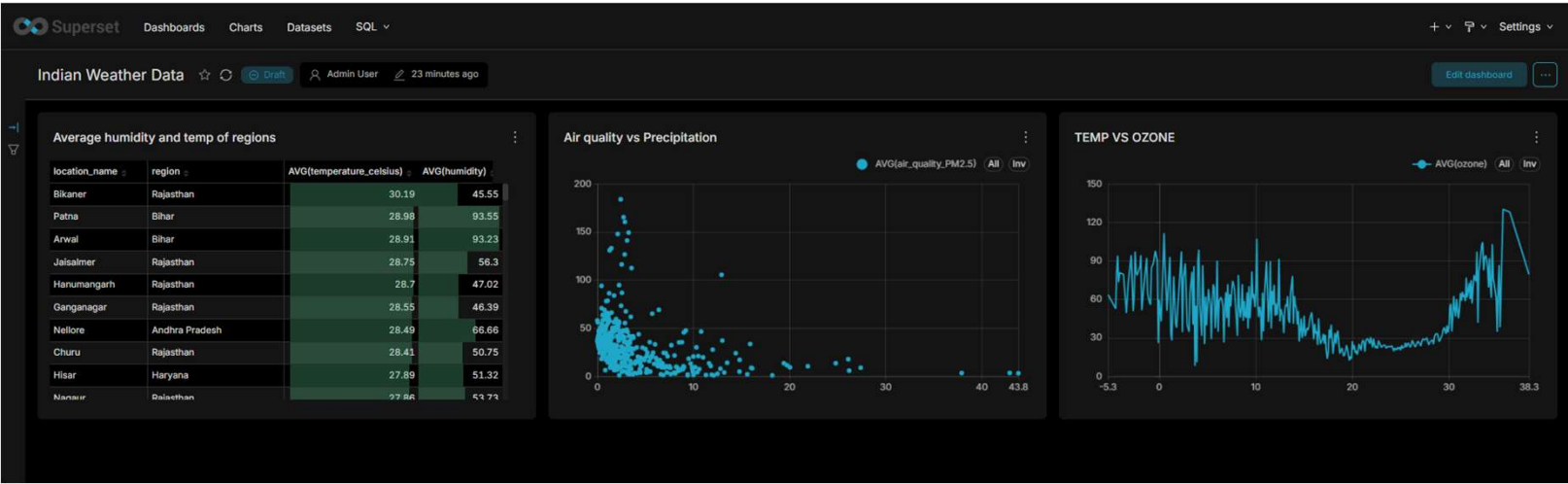
temperature	ozone
38.3	79.4
36.3	114.4
36.3	141.6

Textbox

Ask about your data...

LLM assisted querying:

A diagnostic analysis. The agent investigates whether temperature correlates with ozone, providing a clear 'No' answer, supporting evidence, and a strategic implication—proving it can identify non-obvious patterns.



Manual Querying on database in Superset:

Output of the query given by the LLM was plotted as a line chart between Temperature and Ozone levels in Superset and the visualization was added to the Indian Weather Data Dashboard.

LLM assisted analysis:

This image illustrates complex query synthesis. It demonstrates the tool linking astronomical data with weather metrics, providing an analysis that highlights how moon phases and weather conditions interact.

Complete Indian weather data dashboard:

This is the complete dashboard containing tables generated manually and two tables generated using assistance of the LLM pipeline.

You: Show me the average cloud cover and temperature for each different moon phase. Does the amount of moon illumination seem to change the average temperature?

Analyst Answer:

The average temperature is highest during the Full Moon and New Moon phases (25.83°C vs. 25.71°C), while the Last Quarter phase has the coolest temperature (23.88°C). Cloud cover varies inconsistently with illumination; for example, the New Moon (55.61% cloud) has similar temperatures to the Full Moon but lower cloud cover, and the Waxing Crescent (59.55% cloud) is cooler than both. No clear causal relationship exists between moon illumination and temperature/cloud cover across all phases.

Analysis:

- **Temperature trends:** Higher temperatures occur at Full/Moon (~25-26°C), with a dip at First Quarter (~24°C).
- **Cloud cover variability:** New Moon has the highest cloud cover (55.61%), while Full Moon has low cloud cover (31.71%). No phase consistently pairs lower illumination with higher/cloudiness.

Business Implication:

If decision-making relies on lunar cycles affecting weather, caution is needed due to mixed results. For example, tourism during Full/Moon phases could capitalize on perceived warmth, but agricultural activities might avoid First Quarter for cooler/clement conditions. Data suggests no strong predictive link between illumination and climate, so other factors (e.g., seasonal trends) likely play a larger role. Further analysis with more granular data or statistical testing (e.g., correlation coefficients) would clarify relationships.

Textbox

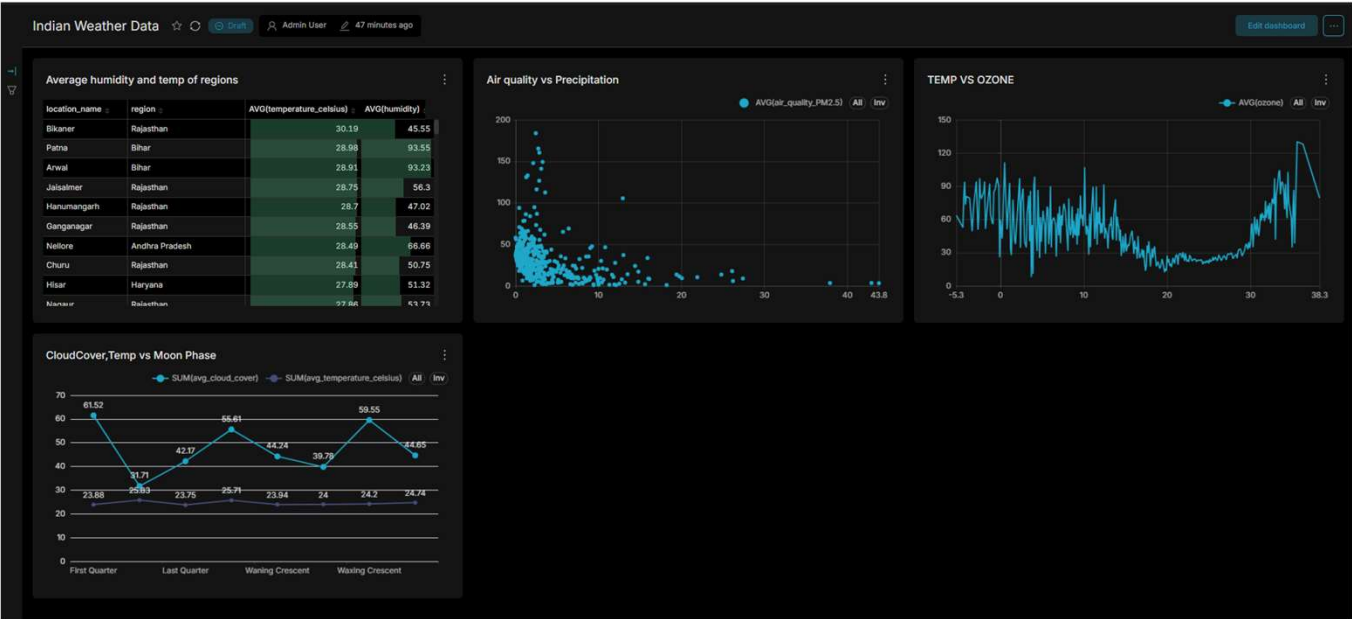
Ask about your data...

Open in Superset SQL Lab: <http://localhost:8080/spillab/>

Table (in DuckDB / Superset)	Rows	Cols
air_quality_information	24,070	9
astronomical	24,070	7
location_information	24,070	8
weather_data	24,070	20

```
SELECT
  a.moon_phase,
  AVG(w.cloud) AS avg_cloud_cover,
  AVG(w.temperature_celsius) AS avg_temperature_celsius
FROM
  astronomical a
JOIN
  weather_data w ON a.last_updated_epoch = w.last_updated_epoch
GROUP BY
  a.moon_phase
```

moon_phase	avg_cloud_cover	avg_temperature_celsius
Waning Gibbous	39.7811	24.0005
Last Quarter	42.1714	23.7476
Full Moon	31.7111	25.8257



Key Takeaways

- Users can go from a raw file to a dashboard chart without writing SQL by hand.
- Every LLM-suggested answer is traceable back to a real, re-runnable Superset query — nothing is a black box.
- The pipeline correctly surfaced non-obvious relationships in the data (e.g., temperature vs. other weather metrics) with a query the user could verify themselves.
- Tested successfully across two very different datasets: Indian weather and Brazilian E-Commerce.